Python

Day-04

07/03/25(F)

In Python, complex numbers are represented using j instead of i because j is the standard notation for the imaginary unit in engineering and computer science.

If you try 3 + 7i, Python will treat it as an invalid expression because i is not recognized as the imaginary unit. Instead, you should use j, like this:

Example:

```
z = 3 + 7j
```

```
print(z)  # Output: (3+7j)
```

If you mistakenly use i, Python will interpret it as a variable name, and unless you define i beforehand, it will throw an error:

Example:

```
z = 3 + 7i  # SyntaxError: invalid syntax
```

So, always use j for complex numbers in Python!

1.Get input for variable name and age.print it.

sample input:

python

1

sample output:

"My Name is:python"

"My Age is:1"

2.Get for variable name and age, address. print it.

sample input:

python

1

Netherland

sample output:

"My Name is:python"

"My Age is:1"

"My Address is:Netherland"

1.Answer:

```
name=input()
age=input()


print("My Name is:",name)
print("My Age is:",age)
```

2.ans

```
name=input()
age=input()
address=input()

print("My Name is:",name)
print("My Age is:",age)
print("My Address is:",address)
```

---

operatots

3.Get integer input for variable a, b, c.

multiply all 3 variables(a*b*c)

add all 3 variable(a+b+c)

divide the multiplied value by added values

print it.

sample input:

2

3

4

sample output: 2.6

outputExplanation:

2*3*4=24

2+3+4=9

24/9=2.6

4.Get input for variable name,score,department

Get score for 100;

divide 100/10

sample input:

Nanisha

80

computer

sample output:

"My Name is Nanisha"

"My score is 8.0"

"My Department is computer"

3.ans

```
a=int(input())
b=int(input())
c=int(input())

d=a*b*c
e=a+b+c
f=d/e
print(f)
```

4. answers

```
name=input()
score=int(input())
department=input()

print("My Name is",name)
print("My score is",score/10)
print("My department is",department)
```

5.rcb="win"

yes we got cup

rcb="lose"

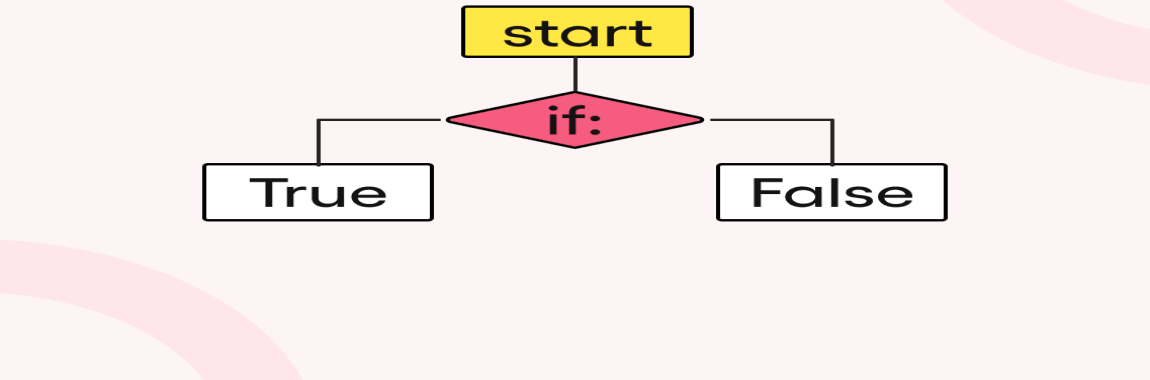
we lose the cup

Python Flow Control

Control Flow Statements in Python

Control Flow Statements in Python are fundamental building blocks that dictate the execution order of a program. They enable developers to create logical pathways and make decisions in their code, using structures like if, for, and while.

These statements are essential for adding complexity and functionality to Python scripts, allowing for conditional execution and repetitive tasks. This introduction will briefly explore how these control flow mechanisms enhance the versatility and efficiency of Python programming.



Reservation Checking in Restaurant

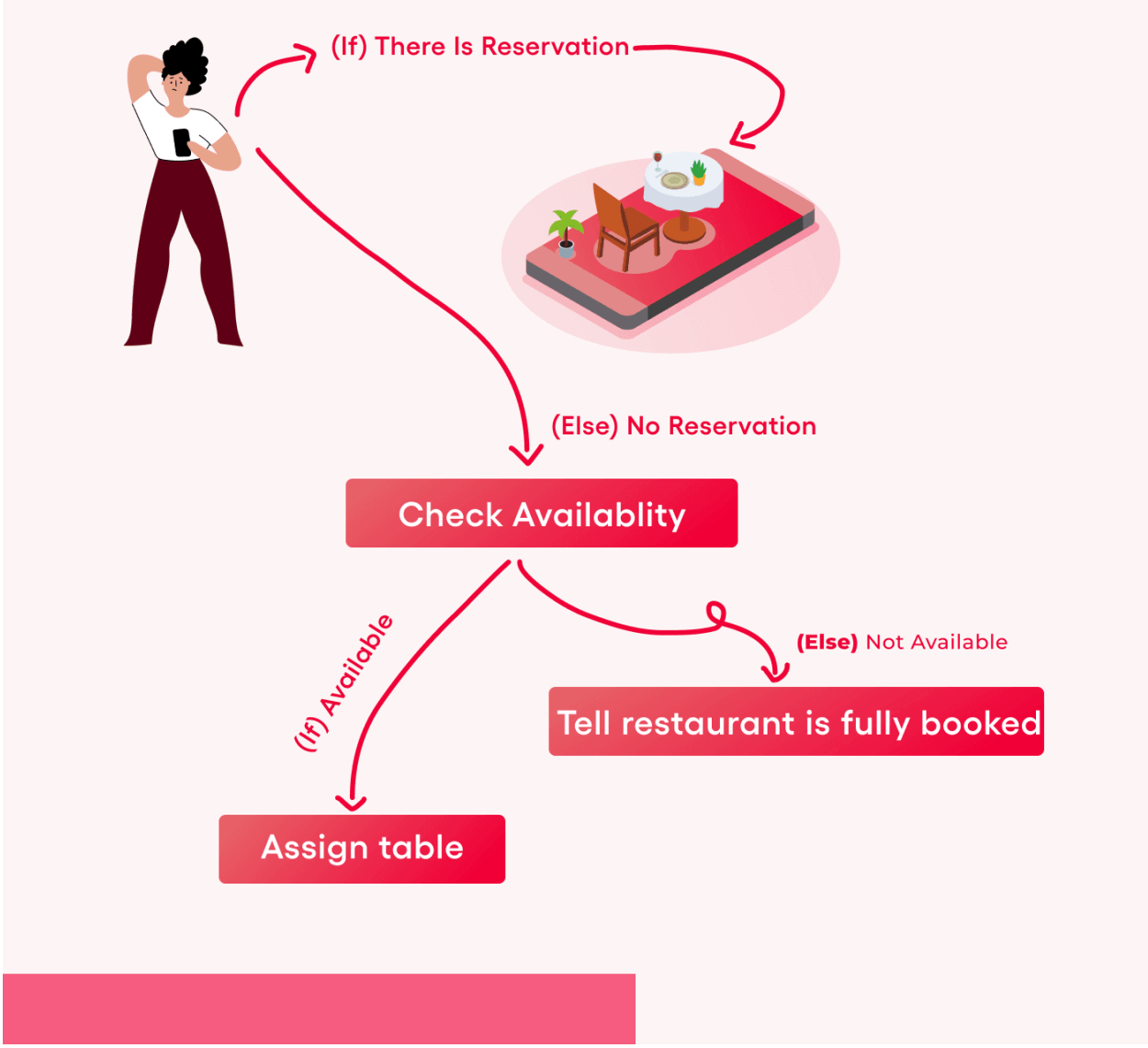
A restaurant manager checks if a customer has a reservation. If the customer has a reservation, they are shown to their table.

❓ If not, the restaurant checks if there are any available tables.

✅ If there are available tables, the customer is seated.

⛔ If not, the customer is told that the restaurant is fully booked and cannot accommodate them.

Let's visualize this example first, then talk about it using Python code.



Python has three types of control structures:

- **Sequential**
- **Selection**

- **Repetition**

Importance of Control Statements in Python

Control flow statements are crucial in Python programming, as they dictate the order in which the code is executed, enabling decision-making, looping, and branching in a program. This makes them indispensable for creating dynamic and efficient software. Here's a brief overview of their importance:

1. Control statements like if, elif, and else allow programs to execute different code blocks based on certain conditions.
2. Loops, including for and while, enables the execution of a code block multiple times.
3. By using control flow statements, programmers can write cleaner, more organized code.
4. Control flow in Python allows for creating efficient algorithms that can make decisions and repeat tasks without human intervention.

Overall, control flow statements are foundational to programming in Python, enabling the creation of versatile, efficient, and intelligent applications. Their proper use is a cornerstone of good programming practices and is essential for leveraging the full potential of Python.

Types of Control Flow in Python

The control flow statements in Python include:

- The if statement
- The if-else statement
- The nested-if statement
- The if-elif-else chain

The if statement in Python

The if statement is a fundamental control flow in Python used to execute a block of code based on a condition. It allows for conditional execution, where the specified code block runs only if the condition evaluates to True. If the condition is False, the code block is skipped, and the program executes the next line of code outside the if statement.

Example:

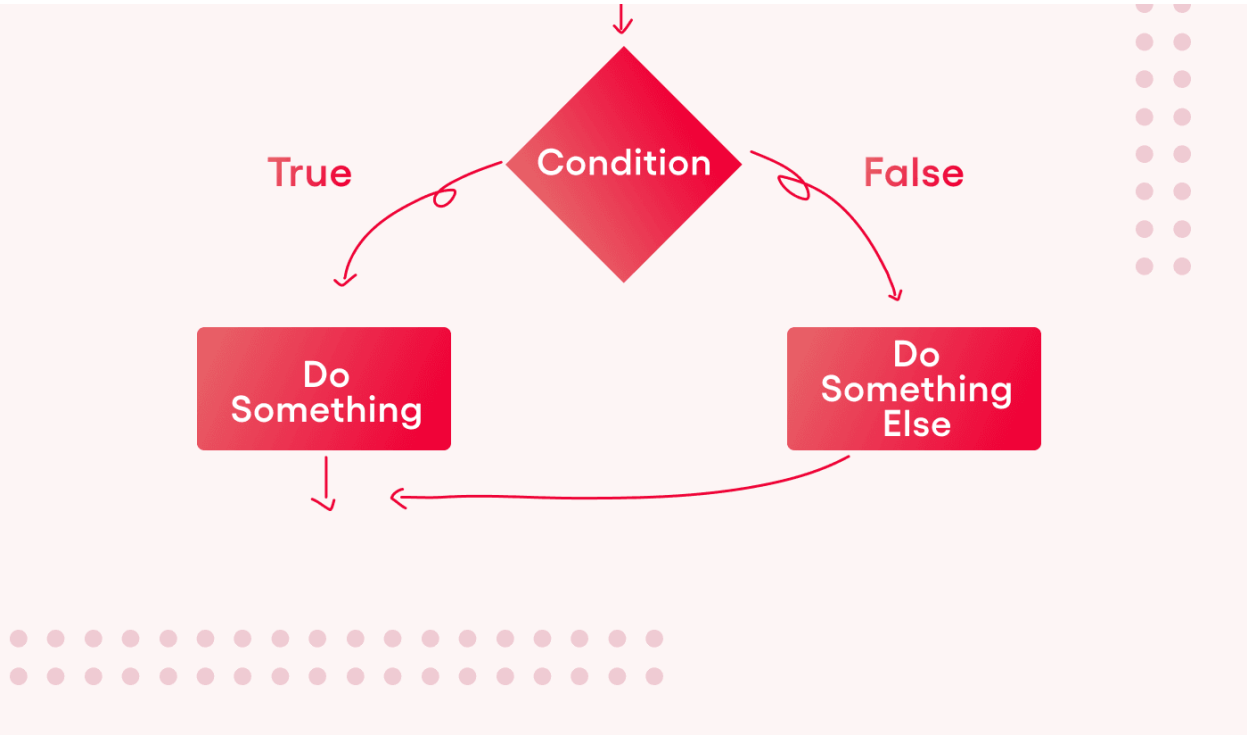
```
x = 10
if x > 5:
    print("x is greater than 5")
```

Output:

```
x is greater than 5.
```

The if-else statement in Python

The if-else statement in Python is a fundamental control flow mechanism that allows for the conditional execution of code blocks. It enables a program to execute specific actions when a specified condition is true and different actions when the condition is false. This bifurcation based on conditions makes the if-else statement a critical tool for decision-making in programming.



Example:

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

Output:

```
x is not greater than 5
```

The nested-if statement in Python

The nested-if statement in Python is a control flow structure that allows you to check multiple conditions sequentially, within other if statements. This nesting capability is handy for handling more complex decision-making scenarios where multiple layers of conditions need to be evaluated.

Example:

```
x = 12
if x > 10:
    if x % 2 == 0:
        print("x is greater than 10 and even")
```

Output:

```
x is greater than 10 and even
```

The if-elif-else ladder in Python

The if-elif-else ladder in Python is a series of conditional statements that provide multiple possible execution paths for a program. It starts with an if condition, followed by one or more elif (short for "else if") blocks to check various conditions in sequence, and ends with an optional else block as a catch-all for any conditions not met by the preceding if or elif blocks.

Example:

```
x = 7
if x > 10:
    print("Greater than 10")
elif x == 7:
    print("x is 7")
else:
    print("Less than 10 and not 7")
```

Output:

```
x is 7
```

Short Hand if-else statement in Python

The shorthand if-else statement in Python, also known as the ternary operator, offers a more concise way to execute simple conditional assignments. This operator allows you to evaluate a condition in a single line, returning one value if the condition is true and another if it's false. It's beneficial for straightforward, inline operations that don't require the full syntax of traditional if-else statements.

Example Let's consider a simple example where we want to assign the string "Adult" to a variable if a person's age is 18 or more and "Minor" if it's less than 18.

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
```

Output:

```
Adult
```

In this example, since age is 20, which satisfies the condition age >= 18, the expression evaluates to "Adult", and thus, the status variable is assigned the value "Adult".

Short Hand if statement in Python

The shorthand if statement in Python allows for a condensed form of writing conditional expressions, primarily used for executing a single action or operation when a condition is true, without providing an alternative for the false case. This form is handy for making code more concise when only a single condition needs to be checked to operate.

Example Consider a scenario where we want to print a message only if a specific condition is met, such as a variable x being greater than 10.

```
x = 15
if x > 10: print("x is greater than 10")
```

Output:

```
x is greater than 10
```

In this example, since x is 15, which satisfies the condition x > 10, the print statement is executed, displaying "x is greater than 10". This illustrates the use of a shorthand if statement to perform a quick, conditional action in a single line of code.

Notes

1. Control flow statements, such as if, if-else, nested-if, and if-elif-else ladders, provide programmers with the tools to handle various decision-making scenarios in their code.
2. Conditional execution and loops helps in writing efficient algorithms that can automate and streamline tasks.
3. Using control flow statements in python helps to keep the code concise and readable.

Conditional Statements (if-elif-else)

1.Write a Python program to check if a number is positive, negative, or zero.

2.Write a program that takes an age as input and checks if the person is eligible to vote (18+).

3.Write a Python program to check whether a given number is even or odd.

4.A shop gives a 10% discount if the purchase amount is more than ₹1000. Write a Python program to calculate the final bill amount.

5.Write a Python program to find the largest of three numbers entered by the user.

1.Ans

```
num = float(input("Enter a number: "))


if num > 0:
    print("The number is Positive.")
elif num < 0:
    print("The number is Negative.")
```

```
else:
    print("The number is Zero.")
```

2.Ans

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

3.Ans

```
num = int(input("Enter a number: "))

if num % 2 == 0:
    print("The number is Even.")
else:
    print("The number is Odd.")
```

4.Ans

```
amount = float(input("Enter the purchase amount: "))

if amount > 1000:
```

```
    discount = amount * 0.10
    final_amount = amount - discount
    print(f"Discount applied: ₹{discount:.2f}")
else:
    final_amount = amount

print(f"Final bill amount: ₹{final_amount:.2f}")
```

5.

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
num3 = float(input("Enter third number: "))

if num1 >= num2 and num1 >= num3:
    largest = num1
elif num2 >= num1 and num2 >= num3:
    largest = num2
else:
    largest = num3

print(f"The largest number is: {largest}")
```

Python if…else Statement

Overview

The decision-making statements are a crucial part of programming as it does give the flow of execution a direction to flow into. Hence, condition checking is the backbone of decision-making, and the python if-else statement becomes handy to help us execute the same. We move forward to decide whether a block of statements will get executed or not. If the if condition around the block of the statements is TRUE, then it will be executed; otherwise, the else block of the statements will be executed.

What Is the Need for if-else Statements in Python

Decision-making statements or conditions in programming languages determine the flow of code execution and its direction. For example, to check whether a number is even or odd and segregate them into two categories, we can use if-else statements in Python. By applying the condition to check for even and odd numbers, we can segregate them accordingly. Python supports various mathematical logics, which are often used while implementing if-else statements to check for conditions in different scenarios.

Various mathematical logics are supported in python, which we often use when we are implementing if else in python to check conditions around scenarios as shown below:

Equals: Scenario where we want to check, when variable a is equal to variable b represented as a == b.

Not Equals: Scenario where we want to check when variable a is not equal to variable b represented as a != b.

Less than: Scenario where we want to check, when variable a is less than variable b is represented as a < b.

Less than or equal to: Scenario where we want to check, when variable a is less than or equal to variable b represented as a <= b.

Greater than: Scenario where we want to check, when variable a is greater than variable b represented as a > b.

Greater than or equal to: Scenario where we want to check, when variable a is greater than and equal to variable b represented as a >= b.

Python if Statement Syntax:

Below is the if-else syntax in python.

if expression

    Statement

else

    Statement

Quick Note: Indentation, the whitespace at the beginning of a line, plays an important role while working with if else in python programming. It is often used to define the scope of the program, while in other programming languages, we often make use of curly brackets to serve the purpose. It is important to take proper care of the indentation while coding with if - elif - else; otherwise, you will encounter IndentationError while executing the code.

Below is an example where proper indentation is not followed in the last print statement. Hence we observe that the code has thrown an error as the compiler considers the last print statement to be out of sync with the above followed indented print statements. This leads to the IndentationError, which could have been avoided if proper indentation had been followed.

What is the need for if-else statements in Python

Various Decision-Making Statements in Python

Below are various combinations of if else in python where we shall study seven methods to implement them in our code when we encounter any decision-making scenario.

If Statement

The simplest decision-making statement used in Python is the if statement. The if statement is significant in Python as it executes the statement in the if block when the condition is true. In Python, all non-zero values are considered true while values like None and 0 are considered false. It is essential to note that it is possible to have a single if statement without an else block, but having an else block without an if statement is not possible in Python.

Syntax:

if expression:

    statement1

statement2

As seen above, the code will evaluate if the expression is TRUE or FALSE. If it is satisfied, then the if statement will execute the block of statements below it; otherwise not. When the test expression is FALSE, then the block of statements is not executed. We can also use condition by using the bracket ( ).

FlowChart: Below is the flowchart for understanding the concept learned above for the if statement as follows:

if FlowChart

The flow of Python program is determined by a test expression, which is validated to decide the program's proper flow. If the test expression is true, the program moves to the body of the if statement and executes the statement block indented below it. It's essential to note that if the statement block isn't properly indented, it will execute regardless of the test expression being true or false. When the test expression in the if statement is false, the program jumps out of the if statement, as shown in the chart.

Code:

```
print(" Case 1 to show how if in python works.")

# print statement


# decalring a variable with its value as 25.

number1 = 25


# if statement

if number1 > 24:

   print(number1, "is a positive number. The `if statement` is satisfied." )

print("I am getting printed as I didn't follow the indentation!")


print(" Case 2 to show when the condition doesn't enter the if in python .")

# print statement

number2 = 99


# decalring a variable with its value as 99.


# if statement

if number2 > 100:

   print(number2,  "is a positive number.The `if statement` is satisfied." )

print("I am getting printed as I didn't follow the indentation!")
```

Output:

Case 1 shows how if in python works.

25 is a positive number. The `if statement` is satisfied.

I am getting printed as I didn't follow the indentation!

Case 2 to show when the condition doesn't enter the if in python.

I am getting printed as I didn't follow the indentation!

If Else

The if statement executes a block of statements only if the condition is true, but if it's false, nothing is executed. To address this, we use the if-else statement in Python, which executes the else statement when the condition is false. Proper indentation is necessary for the if-else statement.

Syntax: The syntax for the if-else statement is as below:

```
if expression1:
   statement1
# When the expression1 is TRUE the statement1 gets executed.
else :
   statement2
# When the expression1 is FALSE the statement2 gets printed.
```

As seen above, the code will evaluate if the expression is TRUE or FALSE. If it does get satisfied, then the if statement will execute the block of statements below it. When the test expression is FALSE, the else statement block of statements is executed. We use indentation to separate the blocks.

FlowChart: Below is the flowchart for understanding the concept learned above as follows: if-else FlowChart The code enters the test expression, and if it's true, it executes the statements in the if block based on indentation. Improper indentation leads to statement execution regardless of the validation. When the test expression is false, the code executes the else block based on indentation.

Code:

```
# Below is the Python Program to understand the concept around the implementation of the If else statement


# print statement
print("Case to show how if else statemen in python works.")


# decalring a variable with  its value as 50.
number1 = 50


# if else statement
if number1 > 24:
   print(number1, "is a greater than 24. The `if statement` is satisfied." )
else:
   print(number1,  "is a lesser than 24.The `else statement` is satisfied." )
print("I am getting printed as I didn't follow the indentation!")



number1 = 23  # decalring a variable with its value as 23.
# if else statement
if number1 > 24:
   print(number1, "is a greater than 24. The `if statement` is satisfied." )
else:
   print(number1,  "is a lesser than 24.The `else statement` is satisfied." )
print("I am getting printed as I didn't follow the indentation!")
```

Output:

Case to show how if else statemen in python works.

50 is a greater than 24. The `if statement` is satisfied.

I am getting printed as I didn't follow the indentation!

23 is a lesser than 24.The `else statement` is satisfied.

I am getting printed as I didn't follow the indentation!

Nested If

In Python, we can use nested if statements to account for multiple possibilities when coding. Nested if statements involve placing an if statement within another if statement. Proper attention to indentation is crucial when implementing nested if statements, especially when dealing with multiple blocks of nested if statements.

Syntax: The syntax for the if statement is as below:

```
if expression1:
   if (expression2):
      statement2
```

When the expression1 is TRUE, then the if statement starts to evaluate the expression2.

The code uses if statements to execute a block of code if a condition is true. It can also contain nested if statements, which evaluate conditions within the initial if statement. The code uses indentation to separate the different blocks of code.

FlowChart: Below is the flowchart for understanding the concept learned above as follows:

Nested if Flowchart

The chart shows how the program's flow is determined by a test expression. If the test expression is true, the program checks a nested test expression. If that is also true, it executes the nested if statements; otherwise, it executes the nested else statements. If the first test expression is false, it executes the else statement below it.

Code:

```
# Below is the Python Program to understand the concept around the implementation of nested if statement


# print statement
```

```
print("Case to show how nested if statement in python works.")

# declaring a variable with its value as 50.
number1 = 50

# nested if statement
if number1 > 20:
   print("Nested if condition 1 is satisfied.")
   if number1 > 25:
      print("Nested if condition 2 is satisfied.")
      print(number1, "is a greater than 20 and 25. The `nested if statement` is satisfied." )
print("I am getting printed as I didn't follow the indentation!")

# decalring a variable with its value as 15.
number1 = 15

# nested if statement
if number1 > 20:
   print("Nested if condition 1 is satisfied.")
   if number1 > 25:
      print("Nested if condition 2 is satisfied.")
      print(number1, "is a greater than 20 and 25. The `nested if statement` is satisfied." )
print("I am getting printed as I didn't follow the indentation!")
```

Output:

```
Case to show how nested if statement in python works.
Nested if condition 1 is satisfied.
Nested if condition 2 is satisfied.
50 is a greater than 20 and 25. The `nested if statement` is satisfied.
```

I am getting printed as I didn't follow the indentation!

I am getting printed as I didn't follow the indentation!

If-elif-else Ladder

The if-elif-else statement checks all if statements in order, and if none of them are true, it moves to check the elif statements. When a condition is true, the block of statements under that if statement is executed, and the rest of the ladder is bypassed. If none of the conditions are true, the else statement is executed.

FlowChart:

Below is the flowchart for understanding the concept learned above as follows: if-else Ladder Flowchart

The chart shows that the program's flow is determined by various test expressions in the if-elif-else statements. If any of the test expressions is true, the block of statements under that if or elif statement is executed. If none of the test expressions are true, the else statement's block of statements is executed.

Syntax: The syntax for the If-elif statement is as below:

```
if expression1:
   print(statement_if)
elif (expression2):
   print(statement_elif)
else:
   print(statement_else)
```

The code evaluates whether the expression is true or false. If it is true, the if statement's block of statements is executed. If the first expression is false, and expression2 is true, the elif statements are executed until one of them is true. If none of them are true, the else statement is executed. Indentation separates the blocks.

Code:

```
# Below is the Python Program to understand the concept around the implementation of the Elif statement


# print statement

print("Case to show how elif statement in python works.")


# declaring a variable with its value as 21.

number1 = 21


# elif statement

if number1 > 20:

   print("if condition 1 is satisfied.")

elif number1 > 25:

      print("elif condition 2 is satisfied.")

else:

   print("else condition is satisfied" )

print("I am getting printed as I didn't follow the indentation!")


# declaring a variable with its value as 28.

number1 = 28


# elif statement

if number1 < 20:

   print("if condition 1 is satisfied.")

elif number1 > 25:

      print("elif condition 2 is satisfied.")
```

```
else:
   print("else condition is satisfied" )
print("I am getting printed as I didn't follow the indentation!")

# declaring a variable with its value as 28.
number1 = 30

# elif statement
if number1 < 20:
   print("if condition 1 is satisfied.")
elif number1 < 25:
      print("elif condition 2 is satisfied.")
else:
   print("else condition is satisfied" )
print("I am getting printed as I didn't follow the indentation!")
```

Output:

```
Case to show how elif statement in python works.
if condition 1 is satisfied.
I am getting printed as I didn't follow the indentation!
elif condition 2 is satisfied.
I am getting printed as I didn't follow the indentation!
else condition is satisfied
I am getting printed as I didn't follow the indentation!
```

Short Hand If

The shorthand if statement is used when we only want a single statement to be executed inside the if block. It is written on the same line as the if statement, and indentation is not important. The shorthand if statement is easier to write and allows the developer to focus on the code rather than proper alignment.

Syntax: The syntax for the Short hand if statement is as below:

if expression: statement

As seen when we are evaluating the shorthand if statement, we always implement it by placing the statement that must get executed after the expression holds TRUE, and this statement must be placed on the same line.

Code:

```
print("Case to show how shorthand if statement in python works.")


number1 = 21



# short hand If  statement

if number1 > 20:     print(number1 , "is greater than 20")

print("I am getting printed as I didn't follow the indentation!")

number1 = 8


# short hand If statement

if number1 > 50:     print(number1 , "is greater than 50")

print("I am getting printed as I didn't follow the indentation!")
```

Output:

Case to show how shorthand if statement in python works.

21 is greater than 20

I am getting printed as I didn't follow the indentation!

I am getting printed as I didn't follow the indentation!

Short Hand If-Else

We implement the shorthand if else statement when there is only one statement that needs to be executed in both the if and else blocks. Here the if-else statements are presented in a single line.

Syntax: The syntax for the Short hand if-else statement is as below:

statement_when_True if expression else statement_when_False

As seen when we are evaluating the shorthand if else statement, we always implement it by placing the statement that must get executed after the expression holds TRUE, and this statement must be placed on the same line. Also, when the condition doesn't hold TRUE, then the else statement will get executed.

Code:

```
# print statement

print("Case to show how shorthand if else statement in python works.")

# declaring a variable with its value as 21.

number1 = 21


# short hand If  else  statement

print(number1 , "is greater than 20") if number1 > 20 else print(number1 , "is lesser than 20")

print("I am getting printed as I didn't follow the indentation!")
```

```
# declaring a variable with its value as 8.

number1 = 8


# short hand If  else statement

print(number1 , "is greater than 50") if number1 > 20 else print(number1 , "is lesser than 50")

print("I am getting printed as I didn't follow the indentation!")
```

Output:

```
Case to show how shorthand if else statement in python works.

21 is greater than 20

I am getting printed as I didn't follow the indentation!

8 is lesser than 50

I am getting printed as I didn't follow the indentation!
```

Conclusion

To make decisions in the python programing language we make use of the conditional statement that is, if-else in python, and related to which we saw various ways to implement the same.

Minimal code is used when we want to execute conditional statements by simply implementing the declaration of all the conditions and statements in a single statemenat while executing the code.

All the combinations that we studied in the article were created by the combinations of if-elif-else lops where:

The if condition is implemented when one of the conditions is true or false, and we want to print the output when our conditions are satisfied.

The Elif condition is implemented when we have a second possibility as the output. Also, we can even have multiple elif conditions to verify for the 3rd, 4th, 5th, and 6th possibilities in our program

The else condition is implemented when we estimate the failure possibility in our code. To avoid encountering any exception or to give the code a solution while encountering a failure possibility, we use this statement.